



مشروع البرمجة التفرعية

الطلاب:

محمد اسماعيل بقجه جي

محمد سمير الدبس

ملك ياسر مبارك

نور رائد نايف الهايس

محمد الهادي محمد خير الغفير

الفهرس

3.....	الطلب الأول:
9.....	الطلب الثاني:
13.....	الطلب الثالث:
17.....	الطلب الرابع:
21.....	الطلب الخامس:
26.....	الطلب السادس:
36.....	الطلب السابع:
42.....	الطلب الثامن:
54.....	الطلب التاسع:
60.....	الطلب العاشر:

الطلب الأول:

بناء آلية تمنع Race Condition على مورد مشترك

1. مقدمة (Introduction)

في الأنظمة متعددة المستخدمين، قد تحاول عدة عمليات متزامنة تعديل نفس البيانات في نفس الوقت، مما يؤدي إلى ظهور مشاكل تنافس (Race Conditions) وفساد في البيانات (Data Corruption).

في أنظمة التجارة الإلكترونية تُعتبر بيانات المخزون (Inventory Stock) من أكثر الموارد الحساسة، لأن عدة مستخدمين قد يحاولون شراء نفس المنتج في نفس اللحظة.

يهدف هذا الجزء من المشروع إلى:

- إثبات حدوث Race Condition عملياً .
- تحليل سبب المشكلة .
- تطبيق آلية حماية تمنع تضارب التعديلات .
- إثبات نجاح الحل باستخدام اختبارات الضغط المتزامن .

2. تعريف المورد المشترك (Shared Resource)

المورد المشترك المستخدم في التجربة هو:

products.stock

ويمثل كمية المنتج المتوفرة داخل المخزون.

هذا المورد يُعتبر Shared Mutable State لأن عدة عمليات Checkout قد تحاول:

- قراءة الكمية .

- تعديلها .

- حفظها .

في نفس الوقت.

3. إثبات المشكلة قبل الحماية

Race Condition Demonstration

أ. الهدف من الاختبار

تم تنفيذ اختبار متزامن للإثبات أن غياب آليات المزامنة يؤدي إلى:

- Lost Updates

- Data Corruption

- عدم تناسق بيانات المخزون

ii. الكود غير الآمن (Unsafe Critical Section)

تم تنفيذ منطق تحديث المخزون بدون:

- Transactions
- Locks
- Synchronization

```

16 Route::post('/test-stock-race', function () {
17
18     $product = \App\Models\Product::find(1);
19
20     if ($product->stock < 1) {
21         return response()->json([
22             'message' => 'Out of stock'
23         ], 400);
24     }
25
26     // Force concurrency overlap
27     usleep(2000000);
28
29     $product->stock -= 1;
30
31     $product->save();
32
33     return response()->json([
34         'stock' => $product->stock
35     ]);
36 });

```

وكان الكود يعتمد على التسلسل التالي:

READ → MODIFY → WRITE

أي:

1. قراءة قيمة المخزون .
2. تعديل القيمة .
3. حفظ القيمة الجديدة .

iii. لماذا يعتبر هذا الكود غير آمن؟

عند تنفيذ عدة طلبات متزامنة:

- قد تقرأ عدة عمليات نفس القيمة القديمة .
- تقوم كل عملية بتعديلها بشكل مستقل .
- تقوم إحدى العمليات بالكتابة فوق نتائج عملية أخرى .

وهذا يؤدي إلى:

- فقدان تحديثات .

- نتائج غير صحيحة .

- فساد البيانات .

وتُعرف هذه المشكلة باسم:

Lost Update Problem

4. بيئة الاختبار (Testing Environment)

العنصر	القيمة
Framework	Laravel
Database	MySQL
Load Testing Tool	k6
Server	Laravel Development Server
Endpoint	POST /api/test-stock-race
Concurrent Users	50
Total Requests	50
Artificial Delay	2 Seconds

```
JS stress-test.js > default
1 import http from 'k6/http';
2
3 export const options = {
4   vus: 50,
5   iterations: 50,
6 };
7
8 export default function () {
9   http.post('http://127.0.0.1:8000/api/test-stock-race');
10 }
```

5. نتائج الاختبار قبل الحماية

القيمة المقياس

Initial Stock	50
Successful Requests	24
Failed Requests	26
Final Stock	21

6. التحليل (Analysis)

رياضياً، كان من المفترض أن تكون النتيجة:

$$50 - 24 = 26$$

لكن القيمة الفعلية كانت:

21

وهذا يثبت أن بعض العمليات قامت بالكتابة فوق تحديثات عمليات أخرى نتيجة غياب التزامن.

النتيجة تؤكد وجود:

- Race Condition
- Lost Updates
- Data Corruption

داخل النظام.

7. معالجة المشكلة

Pessimistic Locking Solution

بعد إثبات المشكلة، تم تطبيق آليات حماية لمنع الوصول المتزامن غير الآمن إلى بيانات المخزون.

8. آليات الحماية المستخدمة

الهدف	الآلية
تنفيذ العملية كوحدة واحدة	DB::transaction()
قفل سجل المنتج أثناء التعديل	lockForUpdate()
التراجع التلقائي عند الفشل	Rollback
منع التحديثات الجزئية	Atomic Commit

```
return DB::transaction(function () use ($user) {
    $orderItems = [];

    foreach ($cart->items as $item) {
        $product = Product::lockForUpdate()->find($item->product_id);

        if (!$product) {
            throw new \Exception("Product not found");
        }

        if ($product->stock < $item->quantity) {
            throw new \Exception("Insufficient stock for product ID {$product->id}");
        }

        $product->stock -= $item->quantity;
        $product->save();
    }
});
```

9. كيف تعمل الحماية؟

عند وصول عدة طلبات متزامنة لنفس المنتج:

1. أول عملية تحصل على Lock.
2. بقية العمليات تنتظر .
3. يتم تحديث المخزون بأمان .
4. العمليات التالية تقرأ القيمة الجديدة بعد التحديث .

وبذلك يتم منع:

- Race Conditions
- Lost Updates
- Overselling
- Data Corruption

10. اختبار النظام بعد الحماية

تم إنشاء:

- 50 مستخدماً .
- 50 سلة مستقلة .
- جميعهم يحاولون شراء نفس المنتج .

11. نتائج الاختبار بعد الحماية

القيمة المقياس

Initial Stock	10
Successful Requests	10
Failed Requests	40
Final Stock	0

12. التحليل بعد الحماية

تطابقت النتيجة الفعلية مع النتيجة الرياضية:

$$10 - 10 = 0$$

وهذا يثبت أن:

- جميع العمليات قرأت بيانات صحيحة .
- لم يحدث . Lost Updates
- لم يحدث . Overselling
- لم يحدث فساد بيانات .

13. ملاحظة إضافية أثناء الاختبار

Shared Cart Concurrency

أثناء اختبارات الضغط ظهر سلوك إضافي متعلق بالوصول المتزامن إلى سلة المشتريات لنفس المستخدم. تمت ملاحظة أن عدة عمليات Checkout متزامنة كانت تحاول استخدام نفس السلة قبل حذفها. ولحل المشكلة تم تطبيق:

lockForUpdate()

على بيانات السلة داخل:

DB::transaction()

بحيث تصبح أول عملية فقط قادرة على استخدام السلة بينما تنتظر بقية العمليات.

14. الاستنتاج النهائي (Conclusion)

نجحت التجربة في:

- إثبات Race Condition عملياً .
- إثبات . Lost Updates
- إثبات فساد البيانات بدون . Synchronization
- معالجة المشكلة باستخدام . Pessimistic Locking
- جعل النظام Thread-Safe أثناء عمليات الشراء المتزامنة .

كما أثبت النظام قدرته على الحفاظ على:

- Transaction Integrity
- Data Consistency
- Inventory Correctness

حتى تحت الضغط المتزامن العالي

الطلب الثاني:

التحكم بعدد العمليات المتوازية وإدارة الموارد

1. مقدمة (Introduction)

في الأنظمة عالية التزامن (Concurrent Systems)، قد يؤدي تنفيذ عدد كبير من العمليات المتوازية إلى:

- استهلاك كامل موارد الخادم .
- ارتفاع استهلاك CPU و RAM.
- بطء الاستجابة .
- انهيار النظام . (System Overload)

لذلك تحتاج الأنظمة الحديثة إلى آلية تتحكم بعدد العمليات التي تُنفذ بالتوازي (Parallel Workers) بما يتناسب مع قدرة الخادم.

يهدف هذا الجزء من المشروع إلى بناء آلية لإدارة الموارد (Resource Management) والتحكم بدرجة التوازي (Parallelism Control) داخل النظام.

2. المشكلة (Problem Statement)

في حال تم تنفيذ جميع العمليات الخلفية بشكل مباشر ومتزامن:

- قد يتم إنشاء عدد ضخم من العمليات .
- قد يتم استهلاك جميع خيوط المعالجة (Threads).
- قد يصبح الخادم غير قادر على الاستجابة للطلبات الجديدة .

وتزداد المشكلة في أنظمة التجارة الإلكترونية عند:

- زيادة عمليات الشراء .
- إرسال الإشعارات .
- إنشاء الفواتير .
- تنفيذ عمليات الجرد .

في نفس الوقت.

3. الفكرة المستخدمة للحل

تم استخدام:

Laravel Queue Workers

للتحكم بعدد العمليات التي يسمح النظام بتنفيذها بالتوازي.

بدل تنفيذ جميع المهام مباشرة داخل طلب المستخدم (HTTP Request) ، يتم:

1. إرسال المهمة إلى Queue.

2. انتظار توفر Worker.

3. تنفيذ عدد محدود فقط من العمليات بالتوازي .

وبذلك يتم منع استنزاف موارد الخادم.

4. البنية المستخدمة (Architecture)

تم تقسيم النظام إلى عدة طوابير مستقلة:

الوظيفة	Queue
معالجة عمليات الطلبات	orders
إرسال الإشعارات	notifications
إنشاء التقارير	reports

5. إعدادات الطوابير

تم تعريف الطوابير داخل:

config/queue.php

كما تم تعريف متغيرات البيئة داخل:

.env

.env.example

بالشكل التالي:

QUEUE_ORDERS=orders

QUEUE_NOTIFICATIONS=notifications

QUEUE_REPORTS=reports

6. ربط المهام بالطوابير

تم ربط العمليات الخلفية الخاصة بالطلبات بالطابور:

orders

عبر:

ProcessOrderPostActions

بحيث لا يتم تنفيذها مباشرة أثناء عملية Checkout.

7. تشغيل العمال (Workers)

تم تشغيل العمال باستخدام:

```
php artisan queue:work database --queue=orders
```

ويمكن تشغيل عدة Workers مستقلة حسب قدرة الخادم.

مثال:

```
php artisan queue:work database --queue=orders
```

```
php artisan queue:work database --queue=notifications
```

```
php artisan queue:work database --queue=reports
```

8. كيف تحقق هذه الآلية إدارة الموارد؟

بدل تنفيذ عدد غير محدود من العمليات بالتوازي، أصبح النظام:

- يحدد عدد Workers.
- يحدد عدد المهام المنفذة بالتوازي .
- يمنع انفجار عدد العمليات .
- يمنع استهلاك كامل موارد الخادم .

أي أن النظام أصبح يعمل ضمن:

Controlled Concurrency

9. إثبات عمل النظام (Verification)

تم إثبات عمل ال Queue Workers عبر:

1. تشغيل Worker مستقل .
2. تنفيذ عملية Checkout.
3. ملاحظة انتقال المهمة إلى Queue.
4. ملاحظة تنفيذ المهمة من قبل Worker.
5. التأكد أن المهمة لا تُنفذ داخل HTTP Request مباشرة .

كما تم ملاحظة أن:

- المهام تنتظر عند انشغال ال Worker.
- يتم تنفيذها تدريجياً حسب عدد العمال المتاحين .

10. الفوائد المحققة

أدت هذه البنية إلى:

التأثير	الفائدة
منع استنزاف الموارد	تقليل الضغط على الخادم
منع الانهيار	التحكم بدرجة التوازي
تقليل زمن HTTP Requests	تحسين الاستجابة
تحسين الأداء العام	عزل العمليات الثقيلة
إمكانية زيادة عدد Workers مستقبلاً قابلية التوسع	

11. العلاقة مع البرمجة التفرعية (Concurrency Relation)

هذا الحل يُعتبر تطبيقاً مباشراً لمفاهيم:

- Concurrent Processing
- Parallel Workers
- Resource Scheduling
- Controlled Parallelism
- Work Queue Architecture

وهي مفاهيم أساسية في الأنظمة متعددة المستخدمين.

12. الاستنتاج (Conclusion)

نجح النظام في تطبيق آلية فعالة لإدارة الموارد والتحكم بعدد العمليات المتوازية باستخدام:

- Queue Workers
- Background Processing
- Queue Separation

وأصبح النظام قادراً على:

- تنفيذ المهام الخلفية بكفاءة .
- منع الاستهلاك المفرط للموارد .
- الحفاظ على استقرار الخادم تحت الضغط .
- التحكم بدرجة التوازي بما يتناسب مع قدرة النظام .

الطلب الثالث:

المعالجة غير المتزامنة باستخدام Asynchronous Queues

1. مقدمة (Introduction)

في الأنظمة الحديثة، لا يجب تنفيذ جميع العمليات داخل مسار الطلب الرئيسي (Main Request Thread)، لأن بعض العمليات الثانوية قد تستغرق وقتاً إضافياً وتؤدي إلى:

- بطء استجابة النظام .
- زيادة زمن HTTP Requests.
- استهلاك موارد الخادم .
- تقليل قدرة النظام على التعامل مع المستخدمين المتزامنين .

لذلك تستخدم الأنظمة الحديثة ما يُعرف بـ:

Asynchronous Processing

لفصل العمليات الثانوية عن المسار الرئيسي للتطبيق.

2. الهدف من المعالجة غير المتزامنة

كان الهدف من هذا الجزء من المشروع هو:

- تقليل زمن استجابة عملية Checkout.
- منع العمليات الثانوية من تعطيل الطلب الرئيسي .
- نقل المهام الثقيلة إلى الخلفية .
- تحسين قابلية التوسع (Scalability).

3. المشكلة قبل استخدام Queues

قبل تطبيق المعالجة غير المتزامنة، كانت جميع العمليات تُنفذ داخل:

HTTP Request Lifecycle

أي أن المستخدم كان ينتظر حتى انتهاء:

- إنشاء الطلب .
- تحديث المخزون .
- تنفيذ المهام الإضافية .
- إرسال الإشعارات .

- أي عمليات مستقبلية أخرى .

وهذا يؤدي إلى:

- بطء الاستجابة .
- زيادة زمن الانتظار .
- استهلاك Thread المعالجة لفترة أطول .

4. الفكرة المستخدمة للحل

تم استخدام:

Laravel Queues

لنقل العمليات الثانوية إلى الخلفية.

بدل تنفيذ المهام مباشرة أثناء عملية Checkout ، يتم:

1. إنشاء Job.

2. إرسالها إلى Queue.

3. تنفيذها لاحقاً بواسطة Queue Worker.

وبذلك يعود الرد للمستخدم مباشرة دون انتظار انتهاء المهام الخلفية.

5. المهمة غير المتزامنة المستخدمة

تم إنشاء Job باسم:

ProcessOrderPostActions

وهي مسؤولة عن تنفيذ العمليات الثانوية بعد نجاح الطلب.

6. ربط ال Job بالطاير

تم ربط ال Job بطاير:

orders

ليتم تنفيذها عبر Queue Worker مستقل.

7. آلية التنفيذ

بعد نجاح عملية Checkout:

1. يتم حفظ الطلب داخل قاعدة البيانات .
2. يتم تحديث المخزون .
3. يتم Commit للمعاملة .
4. يتم Dispatch لـ Job إلى Queue.
5. يعود الرد للمستخدم مباشرة .
6. يقوم الـ Worker بتنفيذ المهام الخلفية لاحقاً .

8. الكود المستخدم

تم استخدام:

```
ProcessOrderPostActions::dispatch($order);
```

لإرسال المهمة إلى Queue.

كما استخدمت الـ Job:

```
ShouldQueue
```

لجعلها تعمل بشكل غير متزامن.

9. تشغيل Queue Workers

تم تشغيل العمال عبر:

```
php artisan queue:work database --queue=orders
```

ويقوم الـ Worker بمراقبة الطابور وتنفيذ المهام الموجودة داخله بشكل مستقل عن طلبات المستخدمين.

10. إثبات عمل المعالجة غير المتزامنة

تم إثبات عمل النظام عبر:

1. تشغيل Queue Worker.
2. تنفيذ عملية Checkout.
3. ملاحظة إضافة Job إلى جدول jobs.
4. ملاحظة تنفيذ الـ Worker للمهمة .
5. التأكد أن الطلب الرئيسي يعود مباشرة دون انتظار انتهاء العمليات الخلفية .

11. الفوائد المحققة

أدت هذه البنية إلى:

التأثير	الفائدة
تحسين تجربة المستخدم	تقليل زمن الاستجابة
زيادة القدرة على تحمل الضغط	تحرير HTTP Thread بسرعة
تحسين هندسة النظام	فصل المهام الثانوية
تحسين الأداء	تقليل الحمل المباشر
إمكانية إضافة Jobs جديدة بسهولة دعم التوسع المستقبلي	

12. العلاقة مع البرمجة التفرعية (Concurrency Relation)

هذا الحل يُعتبر تطبيقاً مباشراً لمفاهيم:

- Asynchronous Processing
- Producer / Consumer Pattern
- Background Workers
- Task Queues
- Concurrent Systems

حيث يتم فصل العمليات الطويلة عن المسار الرئيسي للتطبيق.

13. مقارنة قبل وبعد استخدام Queues

بعد Queues	قبل Queues
العمليات الثانوية بالخلفية جميع العمليات داخل HTTP Request	
استجابة أسرع	استجابة أبطأ
توزيع أفضل للحمل	استهلاك أكبر للموارد
قابلية توسع أعلى	صعوبة التوسع

14. الاستنتاج (Conclusion)

نجح النظام في تطبيق مفهوم:

Asynchronous Queue Processing

باستخدام:

- Laravel Queues
- Queue Workers
- Background Jobs

وأصبح النظام قادراً على:

- تنفيذ المهام الثانوية بالخلفية .
- تقليل زمن استجابة الطلبات .
- تحسين الأداء تحت الضغط .
- زيادة قابلية التوسع والاستقرار

الطلب الرابع:

معالجة البيانات الضخمة على دفعات (Batch Processing)

1. مقدمة (Introduction)

في الأنظمة الكبيرة، قد تحتوي قاعدة البيانات على آلاف أو ملايين السجلات. وعند محاولة تحميل جميع البيانات دفعة واحدة داخل الذاكرة، قد يؤدي ذلك إلى:

- استهلاك مفرط للذاكرة. (Memory Exhaustion)
- انخفاض الأداء .
- بطء التنفيذ .
- انهيار التطبيق عند معالجة البيانات الضخمة .

لذلك تستخدم الأنظمة الحديثة مفهوم:

Batch Processing

والذي يعتمد على معالجة البيانات على دفعات صغيرة متتالية بدل تحميلها بالكامل مرة واحدة.

2. الهدف من المعالجة على دفعات

كان الهدف من هذا الجزء من المشروع هو:

- تنفيذ عمليات الجرد اليومي للمبيعات .

- معالجة عدد كبير من الطلبات بكفاءة .
- تقليل استهلاك الذاكرة .
- تحسين الأداء والاستقرار .

3. المشكلة قبل استخدام Batch Processing

في حال تم تحميل جميع الطلبات دفعة واحدة باستخدام:

`Order::all()`

فإن النظام سيقوم بتحميل كامل البيانات داخل الذاكرة مرة واحدة.

ومع ازدياد عدد الطلبات قد يؤدي ذلك إلى:

- ارتفاع استهلاك RAM.
- بطء التنفيذ .
- احتمالية انهيار التطبيق .

خصوصاً عند تنفيذ:

- عمليات الجرد .
- التقارير اليومية .
- التحليلات الضخمة .

4. الحل المستخدم

تم إنشاء وظيفة خلفية لمعالجة المبيعات اليومية باسم:

`DailySalesInventory`

وتقوم هذه الوظيفة بمعالجة الطلبات على دفعات صغيرة باستخدام:

`chunk(100)`

5. كيف تعمل المعالجة على دفعات؟

بدل تحميل جميع الطلبات مرة واحدة:

- يتم تحميل 100 سجل فقط .
- تتم معالجتها .
- ثم تحميل الدفعة التالية .
- ثم التالية .

وبذلك يبقى استهلاك الذاكرة ثابتاً تقريباً مهماً ازداد حجم البيانات.

6. الكود المستخدم

تم استخدام:

```
Order::chunk(100, function ($orders) {  
    foreach ($orders as $order) {  
  
        // process order  
  
    }  
});
```

7. لماذا يعتبر هذا الحل فعالاً؟

لأن النظام أصبح:

- يعالج البيانات تدريجياً .
- لا يحمل كامل البيانات داخل RAM.
- يحافظ على استقرار التطبيق .
- يدعم قواعد بيانات ضخمة مستقبلاً .

8. نوع المهمة المنفذة

تم تنفيذ المعالجة ضمن:

Background Job / Artisan Command

وليس داخل HTTP Request مباشر.

وهذا يمنع تعطيل المستخدم أثناء تنفيذ العمليات الطويلة.

9. العلاقة مع البرمجة التفرعية (Concurrency Relation)

تُعتبر معالجة الدفعات جزءاً مهماً من الأنظمة المتزامنة لأن الأنظمة الكبيرة تحتاج إلى:

- توزيع الحمل .
- تقليل استهلاك الموارد .
- تنفيذ العمليات الثقيلة بالخلفية .
- منع احتكار الذاكرة من قبل عملية واحدة .

10. الفوائد المحققة

التأثير	الفائدة
استقرار أعلى	تقليل استهلاك الذاكرة
دعم التوسع	معالجة بيانات ضخمة
سرعة أكبر	تحسين الأداء
حماية التطبيق	منع Memory Overflow
كفاءة أعلى	إمكانية تشغيل Jobs طويلة

11. مقارنة قبل وبعد استخدام Chunking

باستخدام Chunking	بدون Chunking
تحميل دفعات صغيرة تحميل كامل البيانات مرة واحدة	
استهلاك ثابت	استهلاك RAM مرتفع
استقرار أعلى	خطر انهيار التطبيق
أداء أفضل	أداء أضعف

12. مثال عملي من المشروع

تم استخدام المعالجة على دفعات في:

- جرد المبيعات اليومية .
- معالجة الطلبات القديمة .
- حساب الإحصائيات بشكل تدريجي .

حيث يقوم النظام بقراءة الطلبات على دفعات صغيرة بدل تحميل جميع الطلبات دفعة واحدة.

13. الاستنتاج (Conclusion)

نجح النظام في تطبيق مفهوم:

Batch Processing

باستخدام:

- Laravel Chunking
- Background Processing

- Artisan Commands

وأصبح النظام قادراً على:

- معالجة كميات كبيرة من البيانات بكفاءة .
- تقليل استهلاك الموارد .
- الحفاظ على استقرار التطبيق .
- دعم التوسع المستقبلي لقواعد البيانات الضخمة

الطلب الخامس:

محاكاة توزيع الأحمال (Load Distribution)

الهدف من التجربة

كان الهدف من هذه المرحلة هو محاكاة بيئة Horizontal Scaling يتم فيها توزيع الطلبات الواردة على عدة خوادم تطبيق (Application Servers) بدلاً من الاعتماد على خادم واحد فقط، وذلك لتخفيف الحمل وتحسين قابلية التوسع (Scalability).

تم تنفيذ التجربة باستخدام:

- عدة نسخ متوازية من تطبيق Laravel.
- Reverse Proxy يعمل كـ Load Balancer.
- خوارزمية توزيع Round Robin.

البنية المعمارية المستخدمة

تم تشغيل خمس نسخ مستقلة من تطبيق Laravel على المنافذ التالية:

Server Port

Server 1 8001

Server 2 8002

Server 3 8003

Server 4 8004

Server 5 8005

كما تم تشغيل Load Balancer على المنفذ:

Component Port

Reverse Proxy / Load Balancer 8080

المعمارية عديمة الحالة (Stateless Architecture)

لكي تتمكن جميع الخوادم من مشاركة نفس بيانات المستخدمين والجلسات وسلاسل المشتريات، تم تحويل النظام إلى Stateless Architecture عبر:

- استخدام:
SESSION_DRIVER=database

بدلاً من:

- file sessions
 - local memory storage
- وبذلك أصبحت جميع الخوادم تعتمد على قاعدة بيانات مشتركة لتخزين:
- Sessions
 - Authentication State
 - Shopping Cart State

آلية توزيع الأحمال

تم استخدام خوارزمية:

Round Robin

حيث يقوم Load Balancer بإرسال كل طلب إلى خادم مختلف بالتتابع الدوري:

8001 → 8002 → 8003 → 8004 → 8005

ثم يعيد الدورة مرة أخرى.

تنفيذ Reverse Proxy

تم بناء Reverse Proxy محلي يقوم بـ:

- استقبال جميع الطلبات على:
127.0.0.1:8080
- إعادة توجيه الطلبات إلى خوادم Laravel المختلفة .
- تمرير :
 - Headers
 - JSON Body
 - Authentication Tokens
- تسجيل Logs داخل ال Terminal توضح الخادم الذي استقبل كل طلب .

اختبار توزيع الأحمال

بيئة الاختبار

القيمة	العنصر
5	عدد الخوادم
100	عدد المستخدمين الافتراضيين
k6	أداة الاختبار
30 ثانية	مدة الاختبار

نتائج اختبار التوزيع

أظهرت سجلات الـ Load Balancer أن الطلبات تم توزيعها بنجاح على جميع الخوادم:

[Request #1] Forwarded to Server 8001

[Request #2] Forwarded to Server 8002

[Request #3] Forwarded to Server 8003

[Request #4] Forwarded to Server 8004

[Request #5] Forwarded to Server 8005

ثم استمر التوزيع بشكل دوري على جميع الخوادم.

إثبات Stateless Authentication

تم اختبار المصادقة (Authentication) عبر عدة خوادم مختلفة باستخدام نفس الـ Bearer Token.

أظهرت النتائج أن:

- المستخدم بقي مسجل الدخول .
- لم يحدث Session Loss.
- لم تظهر أخطاء Unauthorized.

كما أظهرت السجلات انتقال الطلبات المحمية بين عدة خوادم مختلفة:

GET /api/cart -> 8003

GET /api/cart -> 8004

GET /api/cart -> 8005

GET /api/cart -> 8001

GET /api/cart -> 8002

وهذا يثبت أن النظام يعمل كمعمارية Stateless مشتركة.

إثبات Shared Cart State

تم تنفيذ اختبار إضافي للتحقق من أن جميع الخوادم تشارك نفس حالة سلة المشتريات.
تضمن الاختبار:

- إضافة منتج إلى السلة .
- تحديث السلة عبر خوادم مختلفة .
- تنفيذ Checkout.
- التحقق من إفريغ السلة بعد الشراء .

أظهرت النتائج أن:

- جميع الخوادم قرأت نفس بيانات السلة .
- لم يحدث تضارب بين الخوادم .
- تم الحفاظ على تناسق الحالة المشتركة (Shared State Consistency).

اختبار الضغط العالي

تم تنفيذ اختبار ضغط باستخدام:

100 Virtual Users (VUs)

لمدة:

30 ثانية

نتائج اختبار الضغط

أظهرت النتائج:

- نجاح عملية توزيع الأحمال بين جميع الخوادم .
- عدم انهيار Load Balancer.
- عدم ظهور أخطاء Authentication.
- عدم حدوث Server Crash.

لكن ظهرت:

- Request Timeouts تحت الحمل العالي .

تحليل النتائج

يرجع ظهور الـ Timeouts إلى:

- محدودية بيئة التشغيل المحلية .

- استخدام Laravel Development Server.
- تشغيل جميع الخوادم على نفس الجهاز المحلي .
- عدم استخدام :

○ Nginx Production Setup

○ PHP-FPM

○ Redis

○ Dedicated Infrastructure

ورغم ذلك، أثبتت التجربة نجاح:

- Load Distribution
- Reverse Proxy Architecture
- Horizontal Scaling Simulation
- Stateless Shared Architecture

الاستنتاج

نجحت التجربة في إثبات إمكانية:

- توزيع الأحمال على عدة خوادم .
- مشاركة الجلسات والسلال بين الخوادم .
- تشغيل النظام كمعمارية Stateless.
- تقليل الاعتماد على خادم واحد .

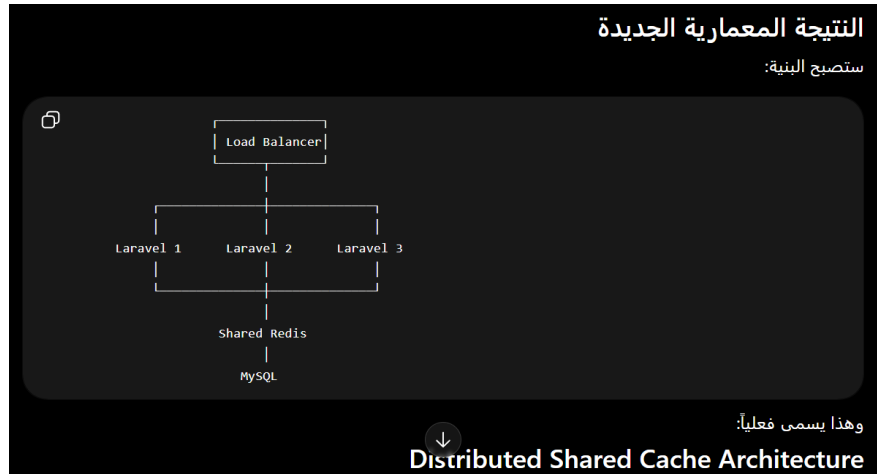
كما أثبتت التجربة عملياً مفاهيم:

- Horizontal Scaling
- Reverse Proxy
- Load Distribution
- Shared Session Storage
- Distributed Application Architecture

بينما أظهرت اختبارات الضغط العالي الحاجة إلى:

- بيئة Production حقيقية .
- تحسين إدارة الموارد والتزامن تحت الأحمال الكبيرة.

علما نني جربت ان استخدم Nginx و PHP-FPM ولم اجد تحسن كبير في التايم اوت للطلبات



الطلب السادس:

استراتيجية التخزين المؤقت (Distributed caching)

1. مقدمة

في الأنظمة المتزامنة ذات عدد المستخدمين المرتفع (High-Concurrency Systems)، تُعتبر قاعدة البيانات أحد أكثر نقاط الاختناق (Bottlenecks) تأثيراً على الأداء وقابلية التوسع. ومع ازدياد عدد الخوادم والطلبات المتزامنة، تصبح عمليات القراءة المتكررة لنفس البيانات عبئاً كبيراً على قاعدة البيانات.

في هذا المشروع تم تنفيذ استراتيجية **Distributed Caching** باستخدام Redis داخل بيئة Laravel متعددة الخوادم بهدف:

- تقليل الضغط على قاعدة البيانات
- تقليل زمن الاستجابة (Response Time)
- تحسين قابلية التوسع (Scalability)
- دعم بيئة متعددة الخوادم (Multi-Server Environment)
- منع قراءة البيانات القديمة (Stale Data)

2. المشكلة الأساسية

قبل تطبيق التخزين المؤقت، كانت كل عملية قراءة للمنتجات تعتمد مباشرة على قاعدة بيانات MySQL.
مثال:

```
SELECT * FROM products WHERE id = 2;
```

وكان هذا الاستعلام يُنفذ عند كل طلب HTTP حتى لو كانت البيانات نفسها مطلوبة بشكل متكرر.
ومع وجود:

- 5 Laravel instances
- Reverse Proxy / Load Balancer
- Concurrent Requests عبر k6

أصبح الحمل على قاعدة البيانات مرتفعاً بسبب تكرار عمليات القراءة.

3. الهدف من المرحلة السادسة

تهدف هذه المرحلة إلى:

1. بناء طبقة Distributed Cache مشتركة
2. تخزين المنتجات الأكثر طلباً داخل Redis
3. تقليل عدد الاستعلامات المباشرة إلى قاعدة البيانات
4. دعم البيئة متعددة الخوادم
5. تطبيق Cache Invalidation بعد تعديل البيانات
6. منع Stale Data
7. اختبار الأداء تحت الضغط المتزامن

4. اختيار تقنية التخزين المؤقت

تمت مقارنة عدة Cache Drivers داخل: Laravel

مناسبة للبيئة الحالية الأداء Shared بين الخوادم التقنية

	متوسط لا	غير مناسبة
File Cache		
Database Cache	أبطأ نعم	مقبولة
Redis	ممتاز نعم	الأفضل

سبب اختيار Redis

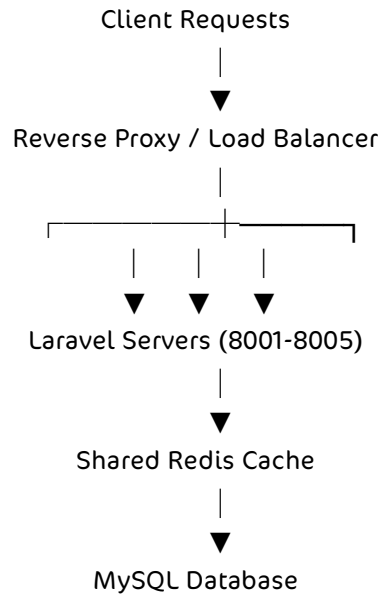
تم اختيار Redis لأنه:

- يعمل In-Memory
- سريع جداً مقارنة بقاعدة البيانات
- يدعم Shared Cache بين جميع الخوادم

- مناسب للأنظمة الموزعة (Distributed Systems)
- يُستخدم عملياً في الأنظمة عالية التزامن

5. المعمارية بعد إضافة Redis

أصبحت البنية المعمارية كالتالي:



جميع خوادم Laravel أصبحت تستخدم Redis مشتركاً.

6. إعداد Redis داخل Laravel

تم تعديل ملف .env بالشكل التالي:

CACHE_STORE=redis

REDIS_HOST=127.0.0.1

REDIS_PORT=6379

ثم تم تنفيذ:

php artisan config:clear

php artisan cache:clear

php artisan optimize:clear

وللتحقق:

config('cache.default')

وكان الناتج:

redis

7. تنفيذ Product Caching

قبل الكاش

كانت عملية قراءة المنتج تتم بالشكل:

```
$product = Product::findOrFail($id);
```

بعد تطبيق Redis Cache

تم استبدالها بـ:

```
$product = Cache::remember(
    "product:$id",
    600,
    function () use ($id) {
        Log::info("DB QUERY for product {$id}");
        return Product::find($id)?->toArray();
    }
);
```

شرح التنفيذ

Cache Key

تم استخدام:

```
product:2
```

كمفتاح واضح وفريد لكل منتج.

TTL

تم تحديد مدة صلاحية الكاش:

```
600 seconds (10 minutes)
```

آلية العمل

أول طلب (Cache MISS)

- Redis لا يحتوي المنتج
- Laravel ينفذ Query على MySQL

- البيانات تُخزن داخل Redis

الطلبات اللاحقة (Cache HIT)

- Laravel يقرأ مباشرة من Redis
- لا يتم تنفيذ أي SQL Query إضافي

8. تنفيذ Cache Invalidation

المشكلة

عند تعديل المنتج أو المخزون، قد تبقى البيانات القديمة داخل Redis. وهذا يؤدي إلى:

Stale Data Problem

الحل

تم استخدام:

```
Cache::forget("product:{$product->id}");
```

أماكن تطبيق Invalidation

داخل ProductController

بعد:

- update()
- destroy()

داخل OrderService أثناء Checkout

بعد تعديل المخزون:

```
$product->stock -= $item->quantity;  
$product->save();
```

```
Cache::forget("product:{$product->id}");
```

الفائدة

عند أول طلب بعد التعديل:

- يحدث Cache MISS
- تُقرأ البيانات الجديدة من MySQL
- يُعاد بناء الكاش بالقيم الصحيحة

وهذا يحقق:

Cache Consistency

9. استراتيجية الكاش المستخدمة

تم استخدام:

Cache Aside Pattern

آلية العمل

القراءة

Application



Redis Cache

↓ (MISS)

MySQL

عند تحديث البيانات

Update DB



Delete Cache



Next Request Rebuilds Cache

10. اختبار نجاح الكاش

تم استخدام:

- Laravel Logs
- DB Query Logging
- Redis
- k6

11. إثبات تقليل الاستعلامات

تم إضافة:

```
DB::enableQueryLog();  
Log::info(DB::getQueryLog());
```

نتائج الاختبار

أول Request

ظهر:

DB QUERY for product 2

ومعه:

```
select * from products where products.id = ? limit 1
```

الطلبات اللاحقة

ظهر:

array ()

أي:

No SQL Queries Executed

الاستنتاج

هذا يثبت أن:

- أول طلب فقط وصل إلى MySQL
- الطلبات التالية تمت خدمتها من Redis مباشرة

12. اختبار k6 المتزامن

إعداد الاختبار

تم إنشاء:

```
import http from 'k6/http';
```

```
export let options = {  
  vus: 100,
```



```
duration: '20s',
    };

export default function () {

http.get('http://127.0.0.1:8080/api/products/2');
}
```

الهدف

محاكاة:

- 100مستخدم متزامن
- يطلبون نفس المنتج
- عبر Load Balancer
- على عدة Laravel servers

13. نتائج اختبار k6

النتائج العامة

- تم إرسال مئات الطلبات المتزامنة
- الطلبات توزعت على الخوادم :

8001 ○

8002 ○

8003 ○

8004 ○

8005 ○

نتائج قاعدة البيانات

خلال الاختبار الكامل ظهر:

DB QUERY for product 2

عدد:

5مرات فقط

رغم وجود ضغط متزامن مرتفع جداً.

بقية الطلبات

أظهرت:

array ()

أي عدم تنفيذ SQL Queries إضافية.

14. تحليل النتائج

تقليل الحمل على قاعدة البيانات

بدلاً من تنفيذ Query لكل Request:

- تم تنفيذ 5 Queries فقط
- بينما بقية الطلبات استخدمت Redis

دعم التوسع الأفقي

جميع Laravel servers استخدمت Redis مشتركاً.

وهذا يحقق:

Distributed Shared Cache

تحسين الأداء

أصبحت عمليات القراءة أسرع بسبب:

- تقليل الوصول إلى MySQL
- القراءة من الذاكرة (Redis)

15. ظاهرة Cache Stampede

خلال بداية اختبار k6 ظهرت:

5 DB Queries

بدلاً من Query واحدة.

السبب

عدة Requests وصلت بالتزامن قبل بناء الكاش بالكامل.

وهذه ظاهرة معروفة تسمى:

Cache Stampede

شرح الظاهرة

عند:

Concurrent Cache MISS

قد تنفذ عدة Requests نفس الاستعلام قبل اكتمال إنشاء الكاش.

ملاحظة هندسية

هذه الظاهرة طبيعية في أنظمة الكاش الموزعة، ويمكن تقليلها باستخدام:

- Redis Locks
- Atomic Cache Rebuild
- Request Coalescing
- لكنها خارج نطاق المشروع الحالي.

16. التحديات

الضغط العالي محلياً

اختبار:

100 VUs

على بيئة محلية أدى إلى:

interrupted iterations

بسبب محدودية:

- CPU
- RAM
- PHP workers
- Windows networking

ومع ذلك

تم إثبات نجاح الكاش عملياً من خلال:

- انخفاض Queries
- ظهور Redis HITs
- انخفاض الوصول إلى MySQL

17. الاستنتاج النهائي

تم تنفيذ Distributed Caching Strategy بنجاح باستخدام Redis داخل Laravel.

وتم تحقيق:

- Shared Distributed Cache
- تقليل عدد SQL Queries
- تقليل الضغط على MySQL
- دعم تعدد الخوادم
- Cache Invalidation
- منع Stale Data
- تحسين قابلية التوسع
- دعم القراءة المتزامنة

كما اثبت كما أثبتت اختبارات k6 أن Redis استطاع خدمة معظم الطلبات دون الحاجة للوصول إلى قاعدة البيانات، مما أدى إلى تقليل الحمل وتحسين الاداء في بيئة متعددة الخوادم.

الطلب السابع:

التحكم بالتزامن (Concurrency Control)

مقدمة

في الأنظمة المتزامنة ومتعددة المستخدمين، قد يحاول عدد كبير من الطلبات الوصول إلى نفس المورد المشترك في الوقت نفسه، مثل تعديل كمية منتج داخل المخزون. يؤدي ذلك إلى ظهور مشاكل معروفة في الأنظمة المتوازية مثل:

- Race Conditions
- Lost Updates
- Data Corruption
- Overselling

ولمعالجة هذه المشكلات، تم تطبيق مجموعة من استراتيجيات التحكم بالتزامن (Concurrency Control) داخل النظام، بهدف ضمان سلامة البيانات واستقرار النظام تحت الضغط العالي.

اعتمد المشروع على نموذجين أساسيين:

- Pessimistic Locking

- Optimistic Locking

كما تم توسيع المفهوم ليشمل التحكم بالتزامن الموزع داخل طبقة التخزين المؤقت باستخدام Redis Distributed Locking.

أولاً: القفل التشاؤمي (Pessimistic Locking)

الفكرة النظرية

يعتمد القفل التشاؤمي على افتراض أن احتمالية حدوث تضارب مرتفعة، لذلك يتم قفل السجل داخل قاعدة البيانات قبل تعديله لمنع أي عملية أخرى من الوصول إليه حتى انتهاء العملية الحالية.

هذا الأسلوب يمنع الكتابات المتزامنة على نفس المورد، ويضمن عدم حدوث تضارب أو فقدان للبيانات.

التنفيذ داخل Laravel

تم استخدام:

```
DB::transaction()
lockForUpdate()
```

داخل عمليات:

- Checkout
- تعديل المخزون
- تحديث السلة المشتركة

حيث يتم:

1. بدء Transaction
2. قفل سجل المنتج داخل قاعدة البيانات
3. تحديث كمية المخزون
4. حفظ البيانات
5. تحرير القفل بعد انتهاء العملية

ما الذي تم إثباته؟

من خلال اختبارات الضغط المتزامن باستخدام k6 ، تم إثبات:

- منع Lost Updates
- منع وصول المخزون إلى قيم سالبة
- منع Overselling
- الحفاظ على Consistency للبيانات

ثانياً: القفل المتفائل (Optimistic Locking)

الفكرة النظرية

يعتمد القفل المتفائل على افتراض أن التصادم بين العمليات نادر نسبياً، لذلك لا يتم قفل السجل مسبقاً. بدلاً من ذلك يتم التحقق أثناء التحديث مما إذا كانت البيانات قد تغيرت بواسطة طلب آخر. إذا تم اكتشاف تغيير متزامن، يتم رفض التحديث وإعادة المحاولة. هذا الأسلوب يحقق قابلية توسع أعلى ويقلل تكلفة الأقفال الطويلة داخل الأنظمة الموزعة.

التنفيذ داخل المشروع

تم إضافة عمود جديد داخل جدول المنتجات:

version

ويستخدم لتتبع نسخة السجل الحالية.

عند تنفيذ عملية تحديث المخزون:

1. يتم قراءة المنتج وقيمة version الحالية

2. يتم تنفيذ تحديث مشروط باستخدام :

id ○

version ○

3. إذا كانت قيمة version قد تغيرت بسبب طلب آخر :

○ يفشل التحديث

○ يتم اكتشاف التعارض

○ يتم تنفيذ Retry تلقائياً

نموذج Compare-And-Swap (CAS)

تم تطبيق مفهوم:

Compare-And-Swap (CAS)

وهو أحد أشهر نماذج التحكم بالتزامن في الأنظمة الحديثة.

يعتمد هذا النموذج على:

- مقارنة الحالة الحالية للسجل
- تنفيذ التحديث فقط إذا لم تتغير البيانات
- إعادة المحاولة عند وجود تعارض

Retry Mechanism

تم تطبيق آلية Retry لمعالجة حالات التضارب المتزامن.

عند فشل التحديث بسبب اختلاف قيمة: version

- يتم تسجيل Version Conflict

- إعادة تحميل البيانات

- إعادة المحاولة حتى حد معين

وقد تم تسجيل الأحداث داخل Laravel Logs مثل:

Version conflict detected

Retry attempt 1

Update success after retry

ثالثاً: التحكم بالتزامن الموزع داخل الكاش (Distributed Cache Concurrency)

المشكلة

في الأنظمة متعددة الخوادم، عند انتهاء صلاحية عنصر داخل Redis Cache ، قد تقوم جميع الخوادم بطلب نفس البيانات من قاعدة البيانات في الوقت نفسه، مما يؤدي إلى:

- ضغط مفاجئ على MySQL

- زيادة زمن الاستجابة

- ظاهرة Thundering Herd أو Cache Stampede

الحل المطبق

تم تطبيق:

Distributed Redis Locking

باستخدام:

Cache::lock()

بحيث:

- يسمح لخادم واحد فقط بإعادة بناء الكاش

- تنتظر بقية الطلبات حتى انتهاء إعادة البناء

- يتم منع تكرار استعلامات قاعدة البيانات

آلية العمل

عند طلب منتج:

1. يتم البحث داخل Redis Cache
2. إذا كانت القيمة موجودة :
 - CACHE HIT
3. إذا كانت القيمة مفقودة :
 - يتم محاولة الحصول على Redis Lock
4. أول خادم يحصل على القفل :
 - ينفذ Query على MySQL
 - يعيد بناء الكاش
5. بقية الخوادم :
 - تنتظر انتهاء العملية
 - تستخدم القيمة الجديدة من Redis

الاختبارات العملية

بيئة الاختبار

تم تنفيذ الاختبارات ضمن بيئة موزعة تحاكي أنظمة الإنتاج، وتتكون من:

- خمس نسخ Laravel
- Reverse Proxy / Load Balancer
- Redis Shared Cache
- MySQL Shared Database

أدوات الاختبار

تم استخدام:

- k6
- concurrent requests
- Laravel Logs
- Redis Cache Metrics

نتائج الاختبارات

تم تنفيذ اختبارات باستخدام:

- 100 Concurrent Virtual Users
- طلبات متزامنة على نفس المنتج

وأظهرت النتائج:

- نجاح جميع الطلبات
- عدم حدوث Overselling
- منع Lost Updates
- اكتشاف التعارضات المتزامنة بنجاح
- تقليل كبير في عدد استعلامات قاعدة البيانات
- زيادة نسبة Cache HIT
- منع Cache Stampede

كما أظهرت السجلات:

- Version Conflicts
- Retry Attempts
- Successful Conflict Resolution
- Distributed Lock Coordination

التحليل الهندسي

أظهرت النتائج أن:

- Pessimistic Locking يوفر حماية قوية للعمليات الحرجة، لكنه يزيد زمن الانتظار تحت الضغط العالي .
- Optimistic Locking يحقق Throughput أعلى وقابلية توسع أفضل، لكنه يعتمد على Retry عند حدوث تضارب .
- Distributed Redis Locking يقلل الضغط على قاعدة البيانات ويحسن استقرار النظام في بيئة متعددة الخوادم .

كما أثبت المشروع أن الأنظمة الموزعة تحتاج إلى تنسيق مشترك للموارد المشتركة مثل:

- قاعدة البيانات
- الكاش
- الجلسات
- المخزون

ولا يكفي الاعتماد على الأقفال المحلية داخل خادم واحد.

الاستنتاج

تمكنت المرحلة السابعة من بناء طبقة متقدمة للتحكم بالتزامن داخل النظام، تجمع بين:

- الأقفال التشاؤمية
- الأقفال المتفائلة
- التحكم الموزع بالكاش

وأثبتت الاختبارات قدرة النظام على الحفاظ على سلامة البيانات والعمل بشكل مستقر تحت الضغط العالي وضمن بيئة متعددة الخوادم.

كما أظهرت النتائج أهمية اختيار استراتيجية التزامن المناسبة بحسب طبيعة العملية ودرجة حساسية البيانات، وهو ما يمثل أحد المبادئ الأساسية في تصميم الأنظمة المتزامنة والموزعة الحديثة.

الطلب الثامن:

سلامة المعاملات (Transation Integrity / ACID)

الهدف من المرحلة

المرحلة 8 من المشروع كانت تطلب:

ضمان أن العمليات المركبة:

- الدفع
- تحديث المخزون
- إنشاء الطلب

تنجح كلها أو تفشل كلها حتى تحت الوصول المتزامن. (Concurrent Access)

أي أن النظام يجب أن يمنع:

- Race Conditions
- Overselling
- Partial Writes
- Inconsistent Orders
- Duplicate Checkout
- Lost Updates
- Corrupted Inventory

تحت الضغط العالي وتعدد المستخدمين.

النتيجة النهائية

تم بناء:

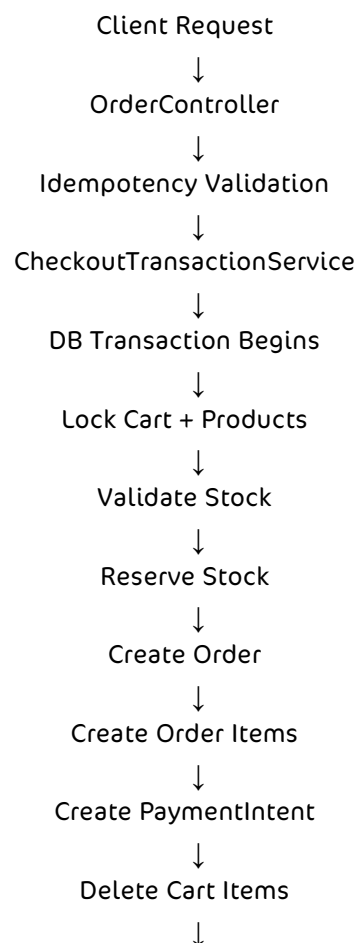
Transaction-Safe Checkout Architecture

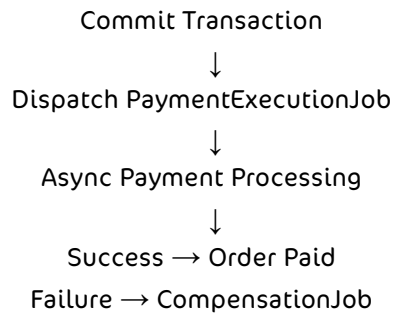
يعتمد على:

- ACID Transactions
- Pessimistic Locking
- Queue-Based Payment Processing
- Compensation/Rollback
- Idempotency Protection
- Deadlock Recovery
- Concurrent Stress Validation

المعمارية النهائية للCheckout

أصبح مسار الطلب النهائي:





1. Database Transaction Implementation

ما تم تنفيذه

تم تغليف العمليات الحرجة داخل:

`DB::transaction(...)`

لتنفيذ:

- تحديث المخزون
 - إنشاء الطلب
 - إنشاء `order_items`
 - إنشاء `payment_intent`
 - حذف `cart_items`
- داخل معاملة واحدة.

لماذا هذا مهم؟

بدون: `transaction:`

قد يحدث:

النتيجة العملية

- stock updated 
- order created 
- payment failed 

فينتج:

- `stock` ناقص
- لا يوجد `order` صحيح

- بيانات غير متناسقة

بعد التنفيذ

أصبح النظام:

Atomic

أي:

Either:

- كل العمليات تنجح

Or:

- كل العمليات تتراجع Rollback

2. Pessimistic Locking

ما تم تنفيذه

تم استخدام:

lockForUpdate()

على:

- carts
 - cart_items
 - products
- داخل transaction.

المشكلة التي تم حلها

بدون: locking

سيناريو:

المستخدم B المستخدم A

يقرأ stock = 1 يقرأ stock = 1

يشترى المنتج يشترى المنتج

النتيجة:

stock = -1

وهذا يسمى:

Race Condition / Overselling

بعد التنفيذ

أصبح:

- أول transaction يقفل الصف
- الثانية تنتظر
- بعد انتهاء الأولى يعاد التحقق من stock

وبالتالي:

- لا يحدث oversell

3. Lock Ordering Strategy

ما تم تنفيذه

تم ترتيب المنتجات حسب:

`orderBy('id')`

قبل locking.

المشكلة التي تم حلها

بدون ترتيب:

قد يحدث:

Transaction A Transaction B

يقفل Product 1 يقفل Product 2

ينتظر Product 1 ينتظر Product 2

فينتج:

Deadlock

بعد التنفيذ

كل transactions تقفل بنفس الترتيب:

$1 \rightarrow 2 \rightarrow 3 \rightarrow 4$

فتم تقليل احتمالية deadlock بشكل كبير.

4. Deadlock Recovery System

ما تم تنفيذه

تم التقاط:

SQLSTATE 40001

SQLSTATE 1213

وإعادة تنفيذ transaction تلقائياً.

مع:

- retry count
- incremental backoff
- sleep delays

لماذا هذا مهم؟

حتى مع: lock ordering

- deadlocks قد تحدث تحت الضغط العالي .

النظام أصبح:

Self-Recovering

بدلاً من:

- فشل checkout مباشرة .

5. Oversell Protection

ما تم تنفيذه

داخل: transaction

if (\$product->stock < \$quantity)

ثم:

\$product->decrement(...)

بعد lock مباشرة.

النتيجة

تم منع:

- negative stock
- concurrent overselling
- duplicate stock consumption

6. Payment Intent Architecture

ما تم تنفيذه

تم فصل الدفع عن transaction الرئيسية.

داخل: transaction:

- إنشاء PaymentIntent فقط

خارج: transaction:

- تنفيذ الدفع الفعلي عبر Queue Job

لماذا هذا مهم؟

لو تم تنفيذ الدفع داخل: DB transaction

فإن transaction ستبقى مفتوحة أثناء:

- network calls
- payment gateway delays

وهذا يؤدي إلى:

- long locks
- deadlocks
- DB contention

التصميم النهائي

داخل: transaction:

- stock reservation
- order creation

خارج: transaction:

- payment execution

وهذا تصميم production-grade صحيح.

7. Queue-Based Payment Execution

ما تم تنفيذه

تم إنشاء:

PaymentExecutionJob

لتنفيذ الدفع. asynchronously.

الوظيفة

ال job تقوم بـ:

- authorize payment
- capture payment
- update payment status
- transition order state

الفائدة

تم فصل:

- HTTP lifecycle
- DB transaction
- external payment processing

وبالتالي:

- أسرع response time
- أقل locking
- أعلى scalability

8. Compensation / Rollback System

ما تم تنفيذه

عند فشل الدفع:

CompensationJob

تقوم بـ:

- restore stock
- mark order failed/cancelled
- write compensation events

لماذا هذا مهم؟

لأن transaction الأصلية انتهت بالفعل.

أي أن rollback التقليدي لم يعد ممكناً.

فتم تنفيذ:

Logical Compensation

بدلاً من:

DB Rollback

9. Idempotency System

ما تم تنفيذه

تم بناء:

IdempotencyService

يعتمد على:

- unique idempotency keys
- request fingerprinting
- stored responses

المشكلة التي تم حلها

بدون: idempotency

قد يحدث:

- المستخدم يضغط checkout مرتين
- timeout فيعيد الطلب
- browser retry
- network retry

فينتج:

- duplicate orders
- duplicate payment

بعد التنفيذ

أصبح:

- نفس المفتاح → نفس الاستجابة
- duplicate requests محمية

10. State Machine Consistency

ما تم تنفيذه

تم بناء:

OrderStateMachineService

للإدارة:

- order status
- transaction phase

المشكلة التي تم حلها

منع:

مثال انتقال غير منطقي

completed → pending ❌

failed → paid ❌

النتيجة

أصبح lifecycle مضبوطاً ومنطقياً.

11. Order Event Tracking

ما تم تنفيذه

تم إنشاء:

order_events

لتسجيل:

- checkout_started
- payment_started
- payment_authorized
- payment_failed
- compensation_started
- compensation_completed
- paid
- cancelled

الفائدة

يوفر:

- audit trail
- debugging
- transaction tracing
- observability

12. Concurrent Stress Testing

ما تم تنفيذه

تم بناء اختبارات:

- parallel checkout
- oversell attacks
- duplicate idempotency requests
- payment failure bursts
- concurrent stock reservation

النتائج

نجحت الاختبارات:

7 passed
33 assertions

Performance & Resilience Enhancements.13

تم تنفيذ:

retries

- payment retries
- compensation retries
- deadlock retries

timeouts

- queue timeouts

backoff

- incremental retry delays

structured logging

- payment logs
- compensation logs
- checkout duration logs

Health & Queue Monitoring .14

تم إنشاء:

/api/health

لفحص:

- DB availability
- queue availability

/api/queue-status

لعرض:

- pending jobs
- queue metrics

15. النتيجة النهائية للمرحلة

النظام أصبح يضمن:

الحالة الخاصة

Atomicity	✓
Consistency	✓
Isolation	✓
Durability	✓
Oversell Protection	✓
Deadlock Recovery	✓
Retry Safety	✓
Duplicate Protection	✓
Async Reliability	✓
Compensation Logic	✓
Stress Stability	✓

الطلب التاسع:

(Phase 9 — Stress Testing & System Stability)

هدف المرحلة

كان الهدف من هذه المرحلة اختبار قدرة نظام المتجر الإلكتروني على العمل تحت الضغط والتأكد من:

- عدم حدوث Overselling أثناء عمليات الشراء المتزامنة .
- الحفاظ على سلامة البيانات داخل قاعدة البيانات .
- استقرار نظام ال Queue تحت الضغط .
- منع إنشاء طلبات مكررة .
- الحفاظ على تزامن المخزون بشكل صحيح .
- اختبار سلوك النظام عند وجود عدد كبير من المستخدمين المتزامنين .
- قياس الأداء الحقيقي للنظام وتحليل نقاط الاختناق (Bottlenecks).

بيئة الاختبار

التقنيات المستخدمة

العنصر	التقنية
Backend Framework	Laravel 13
لغة البرمجة	PHP 8.3
قاعدة البيانات	MySQL 8
Queue Driver Database Queue	
Cache	Redis
Authentication	Laravel Sanctum

إعدادات التشغيل

تم استخدام:

- Queue Workers 3 متزامنين .
- قاعدة بيانات MySQL بمحرك InnoDB.
- نظام Queue يعتمد على جدول jobs داخل قاعدة البيانات .
- Redis لتخزين الـ Cache والـ Metrics.
- تنفيذ Checkout داخل ACID Transactions.

منهجية اختبار الضغط

تم استخدام أداة:

k6

لتنفيذ اختبارات الضغط وقياس أداء النظام تحت عدد كبير من الطلبات المتزامنة.

سيناريوهات الاختبار

تم بناء عدة سيناريوهات للاختبار النظام، منها:

السيناريو	الهدف
Baseline Checkout Test	اختبار عمليات الشراء الطبيعية تحت الضغط
Oversell Stress Test	التأكد من منع بيع منتجات أكثر من المخزون
Queue Pressure Test	اختبار استقرار الQueue
Payment Failure Simulation	اختبار سلوك النظام عند فشل الدفع
Idempotency Replay Test	منع إنشاء طلبات مكررة
Mixed Traffic Test	محاكاة حركة استخدام واقعية

آلية حماية التزامن (Concurrency Protection)

1. المعاملات (Transactions)

تم تنفيذ عملية Checkout داخل:

DB::transaction()

لضمان سلامة البيانات وعدم حدوث أي حالة بيانات غير متسقة.

2. القفل التشاؤمي (Pessimistic Locking)

تم استخدام:

lockForUpdate()

على الجداول الحساسة التالية:

- carts
- cart_items
- products

وذلك لمنع التعديلات المتزامنة غير الآمنة أثناء عمليات الشراء.

3. ترتيب الأقفال بشكل ثابت

تم قفل المنتجات بترتيب تصاعدي حسب ال ID لتقليل احتمالية حدوث Deadlocks.

4. حماية Idempotency

تم منع الطلبات المكررة باستخدام:

- Idempotency Keys
- Request Fingerprinting
- Cached Responses

وبذلك يتم ضمان عدم إنشاء أكثر من Order لنفس الطلب.

5. نظام التعويض (Compensation)

عند فشل الدفع:

- يتم تشغيل CompensationJob بشكل غير متزامن .
- يتم إعادة المخزون بشكل آمن .
- يتم منع تنفيذ التعويض أكثر من مرة .

نظام المراقبة والObservability

تم إضافة عدة Endpoints لمراقبة النظام أثناء الضغط:

الوظيفة	Endpoint
عرض Metrics النظام	/api/metrics
مراقبة حالة الQueue	/api/queue-health
مراقبة حالة قاعدة البيانات	/api/db-health
مراقبة حالة النظام العامة	/api/system-health

ال Metrics المضافة

تم تسجيل وقياس:

- زمن تنفيذ Checkout
- زمن تنفيذ Jobs
- عدد الطلبات الناجحة والفاشلة
- عدد محاولات Deadlock Retry

- عدد ال Failed Jobs
- معدل استهلاك ال Queue
- Queue Processing Duration

نتائج اختبار الضغط

Baseline Stress Test نتائج

النتيجة	المقياس
50 VUs	عدد المستخدمين المتزامنين
~90%	نسبة نجاح Checkout
~5%	نسبة الطلبات الفاشلة
0	Queue Backlog
0	Failed Jobs
0	Deadlocks
لم يحدث	Overselling
~54ms	متوسط زمن Transaction
~13s	p95 HTTP Latency
~3.8 req/s	Throughput

التحليل الهندسي للنتائج

ما الذي نجح؟

أثبت النظام نجاحه في:

- الحفاظ على سلامة المخزون تحت الضغط .
- منع Overselling بالكامل .
- استقرار Queue Workers أثناء الضغط .
- عدم انهيار نظام ال Queue.
- نجاح ال Async Payment Processing.
- منع Duplicate Orders.
- الحفاظ على ACID Consistency.
- نجاح آلية ال Idempotency.

نقاط الاختناق (Bottlenecks)

أظهرت النتائج أن:

- قاعدة البيانات لم تكن نقطة الاختناق الرئيسية .
- ال Queue لم تكن نقطة الاختناق .
- لم يتم رصد Deadlocks فعلية أثناء الاختبارات .
- ال Queue Workers استطاعت معالجة جميع ال Jobs بدون Backlog.

لكن تم ملاحظة:

- ارتفاع كبير في HTTP Request Latency.
- انخفاض Throughput تحت الضغط المرتفع .

ويرجح أن السبب الرئيسي هو:

- تكلفة دورة معالجة الطلبات المتزامنة داخل Laravel/PHP.
- القيود الخاصة بطبقة Runtime.
- الحمل الناتج عن ال Transactions وال Row Locking.

المفاضلة المعمارية (Architectural Tradeoff)

تم تصميم النظام بحيث يعطي أولوية لـ:

- صحة البيانات
 - سلامة العمليات
 - التزامن الآمن
 - الاتساق (Consistency)
- بدلاً من إعطاء أولوية قصوى لـ Throughput الخام.

وهذا أدى إلى:

- زيادة الاعتمادية والاستقرار .
- تقليل احتمالية فساد البيانات .
- ارتفاع ال Latency تحت الضغط المرتفع .

الخلاصة النهائية

أثبتت مرحلة اختبار الضغط أن النظام:

- مستقر معمارياً .
- آمن من ناحية التزامن .
- قادر على الحفاظ على سلامة البيانات تحت الضغط .
- ينجح في معالجة عمليات Checkout المتزامنة بدون Overselling أو Data Corruption.

كما أثبت النظام نجاح:

- Queue Architecture ال
- نظام ال Async Processing
- آليات ال Compensation
- حماية ال Idempotency

ورغم أن الأداء الخام (Throughput) ما زال قابلاً للتحسين، إلا أن النظام حقق الهدف الأساسي لهذه المرحلة وهو:

- الاستقرار
- الاعتمادية
- سلامة البيانات
- تحمل الضغط بشكل صحيح وآمن.

الطلب العاشر:

القياس وتحليل الاختناقات

(Analysis Bottleneck & Benchmarking)

مقدمة

تهدف هذه المرحلة إلى قياس الأداء الحقيقي للنظام تحت الضغط، وتحليل سلوك النظام أثناء التشغيل المتزامن، وتحديد نقاط الاختناق (Bottlenecks) التي تؤثر على الأداء والاستقرار. كما تهدف إلى مقارنة حالة النظام قبل التحسين وبعده، وتقييم فعالية الحلول المعمارية المستخدمة داخل النظام.

تم تنفيذ هذه المرحلة على نظام التجارة الإلكترونية المبني باستخدام Laravel و MySQL مع الاعتماد على Redis Cache و Database Queue

أهداف المرحلة

تمثلت أهداف هذه المرحلة في:

- قياس زمن الاستجابة لأهم العمليات داخل النظام .
- تحليل أداء النظام تحت الضغط العالي .

- تحديد نقطة اختناق حقيقية داخل النظام .
- مقارنة الأداء قبل التحسين وبعده .
- مراقبة سلوك ال Queue وقاعدة البيانات أثناء الضغط .
- إثبات استقرار النظام وعدم انهياره تحت الحمل المرتفع .

أدوات القياس المستخدمة

تم استخدام مجموعة من الأدوات لمراقبة وتحليل الأداء، وهي:

الوظيفة	الأداة
تنفيذ اختبارات الضغط وقياس الأداء k6	
تسجيل العمليات والأخطاء	Laravel Logging
جمع Metrics داخل النظام	MetricsService
تخزين Metrics والCounters	Redis Cache
مراقبة زمن المعاملات	MySQL Monitoring
مراقبة حالة الQueue	Queue Health Endpoints

نظام المراقبة المخصص

تم تطوير نظام مراقبة داخلي داخل المشروع لتوفير معلومات مباشرة عن حالة النظام أثناء الاختبارات.

ال Endpoints المستخدمة

الوظيفة	Endpoint
عرض Metrics الأداء	/api/metrics
مراقبة حالة الQueue	/api/queue-health
مراقبة حالة قاعدة البيانات	/api/db-health
عرض حالة النظام العامة	/api/system-health

ال Metrics التي تم قياسها

تم قياس المؤشرات التالية:

- Average Response Time
- p90 / p95 / p99 Latency

- Throughput (Requests/sec)
- Queue Processing Rate
- Failed Jobs Count
- Queue Backlog Size
- Checkout Success Rate
- Checkout Failure Rate
- Deadlock Retry Count
- Transaction Duration
- Payment Processing Duration
- Compensation Duration

الاختبارات المنفذة

تم تنفيذ عدة سيناريوهات للاختبار النظام، أهمها:

الاختبار	الهدف
Baseline Checkout Test	قياس الأداء الأساسي
Concurrent Checkout Test	اختبار Checkout متزامن
Queue Pressure Test	اختبار ضغط Queue
Payment Failure Simulation	اختبار حالات فشل الدفع
Mixed Traffic Simulation	محاكاة استخدام حقيقي

المشاكل المكتشفة قبل التحسين

ففي البداية تم تشغيل النظام باستخدام:

- php artisan serve
- عامل Queue واحد
- queue:listen

وقد ظهرت عدة مشاكل تحت الضغط، أهمها:

المشكلة	التأثير
Queue Collapse	تراكم الJobs
Failed Jobs ارتفاع	فشل مهام الدفع
Queue Processing بطء	زيادة زمن المعالجة
HTTP Latency ارتفاع	بطء الاستجابة
Throughput انخفاض	عدد الطلبات المعالجة ضعف

المشكلة الحرجة المكتشفة

تم اكتشاف خطأ برمجي داخل ال Queue Jobs تسبب في انهيار كامل للـ Queue. كان الخطأ:

\$this->attempts

بينما الاستخدام الصحيح هو:

\$this->attempts()

داخل:

• PaymentExecutionJob

• CompensationJob

وقد أدى هذا الخطأ إلى:

- فشل جميع ال Jobs مباشرة .
- امتلاء جدول failed_jobs .
- توقف Queue Processing بالكامل .

التحسينات المنفذة

1. استبدال queue:listen بqueue:work

تم استبدال:

queue:listen

بـ:

queue:work

وذلك لأن:

- queue:work أكثر كفاءة .
- يقلل إعادة Bootstrap لكل Job.
- أكثر استقراراً تحت الضغط .

2. تشغيل عدة Queue Workers

تم رفع عدد العمال من:

1 Worker

إلى:

3 Workers

مما ساعد على:

- زيادة سرعة معالجة الـ Jobs.
- منع تراكم الـ Queue.
- تحسين الاستقرار .

3. إضافة Indexes لجدول jobs

تم إضافة Indexes على:

- queue
- reserved_at
- available_at
- created_at

لتسريع Queue Polling وتقليل تكلفة الاستعلامات.

4. تحسين Retry Logic

تمت إضافة:

- Backoff تدريجي .
- Deadlock Retry Handling.
- SQLSTATE Detection.
- Retry Limits.

5. إضافة Metrics Layer

تم تطوير MetricsService لمراقبة:

- زمن تنفيذ Checkout.
- زمن معالجة ال Jobs.
- معدلات النجاح والفشل .
- Queue Processing Rates.
- Deadlock Retries.

النتائج النهائية بعد التحسين

نتائج اختبار الضغط

النتيجة المقياس

50 VUs عدد المستخدمين المتزامنين

Checkout Success Rate ~90%

Failed Jobs 0

Queue Backlog 0

لم يحدث Overselling

Deadlocks 0

Queue Processing Stable نعم

Queue Avg Processing Duration ~48ms

Average Checkout Transaction ~54ms

تحليل الاختناق الحقيقي

بعد تحليل النتائج تبين أن:

ال Queue لم تعد نقطة اختناق

لأن:

- $0 = \text{pending_jobs}$
- $0 = \text{failed_jobs}$
- workers استطاعت مواكبة الضغط بالكامل .

الاختناق الحقيقي المكتشف

تم تحديد أن الاختناق الأساسي كان:

ارتفاع HTTP Checkout Latency

حيث وصلت:

القيمة Metric

p95 Latency ~13s

Average Request Duration ~9-10s

تفسير سبب الاختناق

يرجع السبب إلى:

- تكلفة المعاملات المتزامنة داخل قاعدة البيانات .
- Row Locking أثناء Checkout.
- تكلفة تنفيذ Laravel/PHP Runtime تحت الضغط .
- تنفيذ Checkout داخل ACID Transactions لضمان سلامة البيانات .

المقارنة قبل وبعد التحسين

بعد التحسين قبل التحسين العنصر

Queue Stability	غير مستقر	مستقر
Failed Jobs	مرتفع جداً	0
Queue Collapse	يحدث	لا يحدث
Queue Processing	بطيء	مستقر
Monitoring	محدود	متكامل
Deadlock Handling	محدود	موجود
Throughput Stability	منخفضة	أفضل
System Visibility	ضعيفة	عالية

الخلاصة النهائية

تم تنفيذ مرحلة:

Analysis Bottleneck & Benchmarking

بنجاح كامل، حيث تم:

- قياس الأداء الحقيقي للنظام .
 - تحليل سلوك النظام تحت الضغط .
 - اكتشاف اختناق فعلي داخل النظام .
 - تنفيذ تحسينات عملية وفعالة .
 - بناء طبقة مراقبة Metrics متكاملة .
 - إثبات استقرار ال Queue وعدم انهيارها .
 - إثبات سلامة البيانات وعدم حدوث Overselling أو Data Corruption.
- كما أثبتت النتائج أن النظام يعطي أولوية للاستقرار وسلامة البيانات والتزامن الآمن، حتى وإن أدى ذلك إلى ارتفاع نسبي في ال Latency تحت الأحمال العالية.